

IEOR Visit-Day Matching Tool

Project handoff summary | Master of Analytics Summer Project | June 25, 2026

What the tool does

Prospective graduate students visit the IEO department over two consecutive days and want to meet as many faculty as possible, weighted by research interest. This tool collects faculty availability and student rankings, then builds an optimal meeting schedule that maximizes preference-weighted meetings while keeping the outcome fair across students. It is built for non-technical IEO staff: a tabbed Streamlit app where staff load data, click Match, and review and download per-student and per-faculty schedules.

Current state (working MVP)

- **End-to-end pipeline runs and the optimizer returns OPTIMAL** on synthetic data: roster → form spec → responses → adapter → CP-SAT solver → rendered schedules.
- **Streamlit staff app (app.py)** boots clean (health check passes). Five tabs walk staff left to right: Overview, lay out the two visit days, collect student preferences, collect faculty availability, then build the schedule. Berkeley IEO theme applied.
- **Optimization model (CP-SAT)** prioritizes satisfaction over raw meeting count: a convex rank value (a #1 pick is worth far more than several low picks) plus a max-min fairness floor so no student is starved to feed popular faculty.
- **Synthetic data generator** builds deliberately hard instances (skewed faculty popularity, clustered student interests, availability gaps) off the real 25-faculty IEO roster.
- **Intake scaffolding** for Google Forms / Sheets / Gmail exists and runs in safe preview (dry-run) mode with no credentials; live sending needs a one-time developer setup (SETUP_GOOGLE.md).
- **Pipeline test passes** under both pytest and direct script run.

Repository health and handoff cleanup

The repo is clean and modular: a working tree with sensible commits, a good .gitignore, a solid README, and logic split into focused modules under src/ (config, grid, synthetic, model, adapter, roster, intake). Three issues were found and addressed for handoff:

Issue	Resolution
Stale .git/index.lock from a crashed git process blocks all commits/pulls	Commit/pulls removed from the sandbox. One local command in HANDOFF_GIT.md cleared the lock.
DesignAssets/ screenshots were committed (WebImages/ Assets/ DesignAssets/) and .pytest_cache/ to .gitignore; git rm steps in HANDOFF_GIT.md	Added DesignAssets/ and .pytest_cache/ to .gitignore; git rm steps in HANDOFF_GIT.md
tests/test_pipeline.py ran as a bare script, so pytest collection checks failed; pytest now runs it (1 passed). S	Wrapped collection checks in pytest.main() with a final assert; pytest now runs it (1 passed). S

Two companion docs are in the repo: **HANDOFF_GIT.md** (clear the lock, untrack assets, commit) and **DEPLOY.md** (Streamlit Community Cloud).

Reasonable next steps

Roughly in priority order for the five-person team, toward the August 7 deadline:

- **Wire real intake.** Stand up the Google Forms for faculty availability and student rankings against the schemas in data/, complete the one-time OAuth setup (SETUP_GOOGLE.md), and move the app off synthetic data onto real collected responses.

- **Deploy a shared instance.** Follow DEPLOY.md to put the demo on Streamlit Community Cloud so the team and staff can click a URL instead of running locally.
- **Per-meeting .ics export** for each student and faculty (the ics library is already a dependency).
- **Manual override in the app:** let staff swap or pin a meeting after solving, then re-solve around the locked assignment.
- **Stress-test fairness:** build a deliberately over-subscribed instance and report how the max-min floor allocates the shortfall, so staff understand the tradeoffs the model makes.
- **Harden tests:** add a real pytest suite around the adapter edge cases and the solver feasibility checks so the team can gate changes in CI.

Run locally: `pip install -r requirements.txt` then `streamlit run app.py`. CLI demo: `python src/run.py`.
Tests: `pytest`.